

Guía de Matemáticas para ICPC

Este manual tiene como propósito ser una guía lo más completa posible del material necesario para participar en el Concurso Internacional Universitario de Programación (ICPC, por sus siglas en inglés) en cuanto a la parte matemática se refiere, es cierto que esta competencia es interdisciplinaria y usualmente se busca que los equipos se conformen por gente con un trasfondo fuerte en algoritmia y además en matemáticas. Este libro se enfocará en los conocimientos necesarios de matemáticas que le conviene saber a un concursante por lo que se recomienda para personas estudiantes de matemáticas que quieran iniciarse en la ICPC o personas participantes de ICPC que busquen expandir sus conocimientos matemáticos.

Aunque este manual tenga una gran carga de teoría y definiciones, la familiarización con cada uno de los contenidos de este manual radicará no sólo en leer los capítulos pero también en intentar resolver los problemas y ejercicios sugeridos, si bien algunos problemas fueron recabados de jueces en línea, muchos de ellos fueron diseñados con mucho cariño con la única finalidad de servir para el lector de este manual.

Es importante mencionar que si bien este manual está ordenado por capítulos, estos no están ordenados por dificultad, si no por área de las matemáticas a las que pertenecen, por lo que el lector no necesita necesariamente seguir el orden del índice. El primer capítulo se presentan técnicas de álgebra avanzada usualmente impartidas dentro de los cursos de Álgebra Lineal y Álgebra Moderna, el segundo capítulo se enfocará en la parte de teoría de números, se cubrirán conocimientos impartidos en Teoría de Números I y II, así como propiedades básicas usualmente vistas en la Teoría Analítica de Números. La sección de Combinatoria

ocupará técnicas y propiedades de combinatoria básicas

La segunda parte de este manual esta dedicada a las soluciones de cada una de los problemas presentados en este libro con explicación y código fuente así como algunos hints para poder avanzar en caso de que el lector se encuentre atorado y no quiera leer la solución. Se recomienda al lector realmente intentar resolver los problemas antes de leer las soluciones, pues la parte más valiosa de este manual no es en sí mismo los temas explicados pero la aplicación de cada uno de ellos a los problemas presentados donde las soluciones puede que no requieran temas difíciles pero sí mucho ingenio.

Índice general

1. Introducción a la Programación Competitiva	5
2. Entrada/Salida y Jueces en Línea	13
3. Teoría de números	15
4. Geometría computacional	23
4.1. Bases	24
4.2. Intersecciones	27
4.3. Polígonos	28
4.4. Convex Hull	33
4.5. Algoritmos de barrido	36
4.6. Ejercicios	43
4.7. Problemas	43

Capítulo 1

Introducción a la Programación Competitiva

El sitio oficial del Concurso Internacional Universitario de Programación (ICPC, por sus siglas en inglés) menciona que la ICPC es “un concurso de programación algorítmica para estudiantes universitarios”, aunque esta definición es correcta, si el lector nunca ha participado en esta, es probable que no tenga clara cuál es la diferencia entre esta competencia y otras que existen en el mundo de la programación, por ejemplo hackatones famosos como “Microsoft Global Hackaton” o “NASA Space Apps”, en los cuáles se pone énfasis en desarrollar soluciones a problemas también. La diferencia radica principalmente en que en las últimas se pone mucho énfasis en desarrollar proyectos completos o aplicaciones durante varios días que resuelven un gran problema, generalmente del mundo real y abierto a que

los concursantes exploren diferentes perspectivas del mismo, por ejemplo en “NASA Space Apps 2025” uno de los problemas a resolver era el de buscar exoplanetas usando inteligencia artificial.

Por otro lado en la ICPC y otras olimpiadas de programación lo que se busca es que los concursantes programen soluciones a problemas concretos, en los que la correctez de una solución se basa en si esta arroja la respuesta correcta a todos los casos de prueba propuestos, esto es evaluado por un juez, el cuál se explicará cómo funciona en el siguiente capítulo.

La competencia más importante de programación competitiva a nivel universitario actualmente es la ICPC, esta es una competencia por equipos de 3 personas ¹, que se realiza de forma anual con varias fases, en caso de México, desde el año 2024 se introdujo una nueva fase, de forma que actualmente consta de 4 etapas. La primera etapa es el Gran Premio de México, el cuál consiste en 3 fechas, en las que en cada una se dejan alrededor de 9 a 13 problemas que como se verá a lo largo de este texto abarcan diferentes temas. Los equipos tienen 5 horas para poder resolver la mayor cantidad de problemas posible, es importante mencionar que si 2 o más equipos resuelven la misma cantidad de problemas el orden de la tabla se decidirá por qué equipo tiene menor penalización, la cuál es la suma de los tiempos, en minutos, en los cuales se resolvió cada problema, cada intento incorrecto sumará 20 minutos extra de penalización al equipo en caso de resolver el problema del que fue el envío incorrecto.

¹Estos deben ser estudiantes universitarios de la misma institución

Tras las 3 fechas los cupos para clasificar al regional, también llamado “ICPC Mexico Finals” se asignan por varias categorías ², es importante mencionar que la cantidad de cupos de cada tipo así como sus reglas suelen variar año con año, se recomienda consultar el sitio oficial constantemente.

Ahora hablaremos de cómo se ve un problema en la ICPC y qué formato tiene, es importante tomar en cuenta las siguientes partes:

- **Tiempo Límite:** Es el máximo tiempo que puede tardar el programa en ejecutarse. Si se excede este límite, la solución será incorrecta aún cuando la lógica esté bien, esto intenta prevenir que se manden soluciones poco eficientes.
- **Memoria Limite:** Es la memoria RAM máxima que puede usar la solución enviada.
- **Descripción:** Aquí se da contexto del problema. Muchas veces se incluye una narrativa, sin embargo ya que el tiempo es limitado se recomienda identificar la tarea a resolver.
- **Input (Entrada):** Explica el formato en el cuál el programa solución deberá recibir los datos. Incluye tipo de datos y restricciones.
- **Output (Salida):** Especifica qué debe imprimir la solución enviada. Incluye especificaciones de formato, número de líneas, espacios, así como precisión si son números flotantes.

²Ver Anexo 1.1

- Ejemplos: Aquí se dan algunos casos de entrada con su respectiva salida para ayudar al participante a aclarar dudas.

Veremos un problema de ejemplo del juez en línea Codeforces ³:

[1.1] Ejemplo

Un día 3 mejores amigos Petya, Vasya y Tonya decidieron formar un equipo y participar en concursos de programación competitiva. Mucho antes del inicio los amigos decidieron que implementarían un problema si al menos dos de ellos estaban seguros de la solución. De lo contrario, los amigos no escribirán la solución del problema.

El concurso consiste de n problemas. Para cada problema, sabemos qué amigo está seguro de la solución. Ayuda a los amigos a saber la cantidad de problemas para los que si escribirán la solución.

Input:

La primera línea de entrada contiene un único número entero n ($1 \leq n \leq 1000$) — el número de problemas en el concurso. Las siguientes n líneas contienen tres números cada una, cada número entero es 0 o 1. Si el primer número es 1, entonces Petya está seguro de la solución del problema, de lo contrario no está seguro. El segundo número muestra la opinión de Vasya sobre la solución, el tercer número muestra la opinión de Tonya. Los números en cada línea están separados por espacios.

Output:

Imprima un sólo número entero, la cantidad de problemas

³codeforces.com/problemset/problem/231/A

que los amigos implementarán en el programa.

Ejemplos:

Entrada	Salida
3 1 1 0 1 1 1 1 0 0	2
Entrada	Salida
2 1 0 0 0 1 1	1

Notemos que en este problema lo que tenemos que hacer es contar en cada línea de la entrada cuántos amigos están seguros de la solución, es decir, cuántos 1 hay, en caso de ser 2 o más sumamos uno a la cuenta de los problemas a implementar, de forma que el código solución quedaría así:

```
1 #include<bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     int n;cin>>n;
6     int implementados = 0;
7     for(int i = 0; i < n; i++){
8         int contador = 0;
9         for(int j = 0; j < 3; j++){
10             int temp; cin>>temp;
11             if(temp == 1){
12                 contador++;
13             }
14         }
15         if(contador >= 2){
16             implementados++;
17         }
18     }
```

```
19     cout<<implementados<<"\n";  
20 }
```

Ejercicios

[1.2] Un día caluroso de verano, Pete y su amigo Billy decidieron comprar una sandía. Cuando pesaron la sandía, la báscula indicaba w kilos. Como tenían sed, se apresuraron a llegar a casa y decidieron dividir la sandía, pero se encontraron con un problema difícil.

Pete y Billy son grandes fanáticos de los números pares, por lo que quieren dividir la sandía en dos de forma que ambas partes pesen una cantidad par de kilos, no importa si estas no pesan lo mismo. Los niños están muy cansados y quieren empezar a comer lo más pronto posible, por eso te pidieron ayuda para saber si pueden dividir la sandía de la forma que ellos quieren. Ambos niños deben recibir una cantidad positiva de peso.

Input:

La primera (y única) línea de entrada contiene un número entero w ($1 \leq w \leq 100$) — el peso de la sandía que compraron los niños.

Output:

Imprima YES, si los niños pueden dividir la sandía en dos partes, cada una de ellas con una cantidad par de kilos; y NO en el caso contrario.

Ejemplos:

[1.3] Te dan dos enteros, a y b . De todos los valores posibles de c tal que $a \leq c \leq b$ encuentra el valor mínimo de $(c - a) + (b - c)$.

Entrada	Salida
8	YES

Input:

La primera línea contiene un entero $1 \leq t \leq 55$ que indica la cantidad de casos. La segunda línea contiene dos enteros a y b , tal que $1 \leq a \leq b \leq 10$.

Output:

Para cada caso imprimir el mínimo valor de $(c - a) + (b - c)$.

Ejemplos: [1.3] Hay un nuevo videojuego que se llama

Entrada	Salida
3	1 7 0
1 2	
3 10	
5 5	

“Quiero ser el sujeto” que consiste en n niveles. Little X y su amigo Little y son adictos al juego. Cda uno de ellos quiere pasarse todo el juego. Little X solo puede pasara p niveles del juego y Little Y solo puede pasarse q niveles. Te van a dar los indices de los niveles que puede pasar cada uno, ¿pueden Little X y Little Y pasarse todo el juego si cooperan juntos?

Input:

La primera línea contiene un entero $1 \leq n \leq 100$. La siguiente línea contiene un entero $p(0 \leq p \leq n)$ seguido por p diferentes enteros $a_1, a_2, \dots, a_p$ ($a \leq a_i \leq n$). Estos enteros denotan los índices de niveles que Little X puede pasar. La siguiente línea contiene los niveles que Little Y puede pasar en el mismo formato. Se asume que los niveles están numerados de 1 a n .

Output:

Si pueden pasar todos los niveles, imprime "I become the guy.". Si es imposible, imprime "Oh, my keyboard!".

Ejemplos:

Entrada	Salida
4 3 1 2 3 2 2 4	I become the guy.
4 3 1 2 3 2 2 3	Oh, my keyboard!

Capítulo 2

Entrada/Salida y Jueces en Línea

En el capítulo anterior mencionamos los jueces en línea (online judges), a grandes razgos, estos son sistemas que reciben un programa enviado por un usuario, entonces ejecutan este con entradas predefinidas a las que se les llama casos y compara las respuestas arrojadas por el programa con las esperadas, y a partir de esto determina si la solución enviada es correcta o no.

El funcionamiento de un juez en línea se puede ver de la siguiente manera:

Cliente → Frontend → Backed → Evaluador

Capítulo 3

Teoría de números

En programación competitiva, el dominar o al menos tener buenas nociones de los conceptos principales de teoría de números no es un lujo, es una necesidad, el lector se dará cuenta a lo largo de este texto y en su aventura resolviendo problemas en la ICPC que muchos de los algoritmos usan alguna noción o idea de teoría de números.

Algoritmos básicos

Dados dos números a y b , nuestra tarea será encontrar el mayor número que es divisor tanto de a como de b , por ejemplo, si $a = 6$ y $b = 4$ tendríamos que el mcd (maximo común divisor) sería 2. Ahora ya que no es práctico intentar todos los divisores, presentaremos un algoritmo llamado **algoritmo de euclides** que es más práctico para encontrar este número. Este algoritmo se centra en la afirmación de que dados dos números a y b , de forma de b no divida a a , si escribimos $a = bq + r$ entonces $\text{mcd}(a, b) = \text{mcd}(b, r)$, de

aquí que podamos escribir lo siguiente:

$$a = bq + r_1$$

$$b = r_1q_1 + r_2$$

$$\vdots$$

$$r_{n-1} = r_nq_n$$

y por la afirmación anterior, $\text{mcd}(a, b) = \text{mcd}(b, r_1) = \text{mcd}(r_1, r_2) = \dots = r_n$, nótese que además $r_1 = a \bmod b$, de forma que algorítmicamente tendríamos que el $\text{mcd}(a, b) = a$ si $b = 0$, de lo contrario $\text{mcd}(a, b) = \text{mcd}(b, a \bmod b)$ y así podemos escribir el siguiente código:

```
1 int mcd(int a, int b) {  
2     if(b==0)  
3         return a;  
4     else  
5         return mcd(b, a % b );  
6 }
```

Esta función no es necesario implementarla pues está incluido como función estándar desde C++ 17 con el nombre de *gcd* (greatest common divisor).

Además de obtener este código, este algoritmo nos dice que $\text{mcd}(a, b) = ax + by$ para algunos enteros x, y , es decir, $\text{mcd}(a, b)$ se puede ver como combinación lineal de a y b , esto se le conoce como el lema de Bezout.

Otra tarea que nos podría parecer interesante es dados dos números a y b , cómo calcular el mínimo común múltiplo (denotado *mcm*), esto es, el número menor que es divisible por tanto a como b . Este problema en realidad

está casi resuelto con nuestro código de arriba, esto es por una propiedad importante del *mcd*:

$$mcm = \frac{ab}{mcd(a,b)}$$

de forma que nuestra implementación sería simplemente calcular la fórmula de arriba, sin embargo aquí un detalle importante a considerar es que si nuestros enteros son muy grandes, puede que tengamos un overflow al multiplicar *ab*, de forma que podemos primero dividir y después multiplicar, tomar en cuenta que decisiones de esta índole aunque simples pueden evitarnos veridictos incorrectos en muchos problemas:

```
1 int mcm(int a, int b){  
2     return (a/mcd(a,b))*b;  
3 }
```

Aritmética modular

Iniciemos analizando un ejemplo.

Ejemplos [3.1] Supongamos que hoy es Lunes, ¿qué día de la semana será dentro de 300 días?

Solución. Podríamos hacer una lista de 300 números después del lunes, asignando cada día a cada número y ver cuál queda en el número 300, pero esto sería muy tardado y realmente innecesario, pues si notamos que cada 7 días se repite el mismo día, podemos ver que ya que al dividir 300 entre 7 nos da 42 y nos queda de residuo 6, por lo que sabemos que el día 264, es decir 42 veces 7, será Lunes y sólo tenemos que ver los restantes 6 días, que sabiendo que 6 días después de Lunes es Domingo, tendremos nuestra respuesta.

En este ejemplo vemos como la noción de que conjuntos

de números se conformen de forma cíclica nos simplifica problemas. Si en el ejemplo anterior en lugar 300 días fueran 314 días, la respuesta sería la misma, podría parecer una afirmación fuerte, pero realmente al dividir 314 entre 7 como hicimos en el ejemplo notaremos que el residuo es el mismo, es decir 6, entonces el día 308, es decir 44 veces 7, será Lunes y tenemos nuevamente que ver los siguientes 6 días, esto nos lleva a pensar que si de días de la semana estamos hablando, dos días representan el mismo día de la semana si y sólo si tienen el mismo residuo al dividirlos entre 7.

Resulta que esto mismo lo podemos hacer con cualquier número natural n , sea n un número natural. Si x y y son dos números enteros tales que tienen mismo residuo al dividirlos entre n diremos que x es congruente a y módulo n lo que podemos escribir con la siguiente notación:

$$x \equiv y \pmod{n}$$

Antes de ver ejemplos, veamos algunas propiedades de las congruencias, si $a \equiv b \pmod{n}$ y $c \equiv d \pmod{n}$ y m es un entero positivo, entonces lo siguiente se cumple:

$$a + c \equiv b + d \pmod{n}$$

$$ac \equiv bd \pmod{n}$$

$$a^m \equiv b^m \pmod{n}$$

Como podemos notar, las congruencias respetan las sumas, productos y potencias como estamos acostumbrados, y en realidad para sumas y productos ocurre que cualquier número puede sustituirse por otro al que sea congruente en el módulo de la congruencia.

Inverso Multiplicativo Modular

Aunque pueda parecer que las operaciones con congruencias son muy flexibles, hay que tener cuidado, pues aún no hemos mencionado nada sobre la división. Suena natural querer definir un número que sea un inverso multiplicativo, es decir dado un número a y un número n , encontrar un número b tal que $ab \equiv 1 \pmod{n}$, a este lo denotaremos como a^{-1} .

Es importante mencionar que este inverso no siempre existe. Tomemos por ejemplo el caso de $n = 4$ y $a = 2$, es fácil ver que revisando todas las posibilidades de b módulo n no existe un número que cumpla lo pedido. Se puede probar que este inverso existe si y solo si $\text{mcd}(a, n) = 1$.

Ahora, también podemos diseñar un algoritmo para encontrarle en caso de que exista, recordemos que arriba en esta sección vimos el algoritmo de euclides para poder encontrar el mcd de dos números, y en particular teníamos que $\text{mcd}(a, b) = \text{mcd}(b, r_1) = \dots = \text{mcd}(r_{n-1}, r_n) = \text{mcd}(r_n, 0) = r_n$, donde los r_i eran los residuos, también por el lema de Bezout tenemos que existen x y y tal que $\text{mcd}(a, b) = ax + by$, pues bien, queremos encontrar esos coeficientes y notemos que tenemos ya una combinación lineal trivial con r_n y 0 de la que conocemos los coeficientes, pues de lo anterior

$$\text{mcd}(a, b) = \text{mcd}(r_n, 0) = 1r_n + 0$$

entonces intentaremos usar ingeniería inversa de las ecuaciones del algoritmo de euclides para encontrar estos x y y para a y b iniciales, es decir si antes teníamos que en cada caso a y b pasaban a ser b y $a \bmod b$, ahora queremos ver conociendo x_i y y_i en una ecuación $x_i b + y_i a \bmod b = \text{mcd}(a, b)$ quiénes son x y y en la ecuación $xa + yb = \text{mcd}(a, b)$, nóte-

se que ya que $a \bmod b = a - (\lfloor \frac{a}{b} \rfloor * b)$ podemos sustituir y tenemos que

$$\text{mcd}(a, b) = x_i b + y_i (a - (\lfloor \frac{a}{b} \rfloor * b)) = ay_i + b(x_i)$$

Sin embargo, aunque el algoritmo anterior funciona, para algunos problemas será importante que podamos calcular este inverso en tiempo lineal, entonces presentaremos un método usando exponenciación binaria

Métodos de factorización

Ejercicios

[3.1] Definamos $S(x)$ como la suma de dígitos de un número x en sistema decimal. Por ejemplo $S(11) = 2$, $S(1002) = 3$, $S(5) = 5$.

Diremos que un entero x es interesante si $S(X+1) \leq S(X)$. En cada test se te dará un número n . Tu tarea es calcular cuántos números x cumplen que $1 \leq x \leq n$ y x es interesante.

Input:

La primera línea contiene un entero t ($1 \leq t \leq 1000$) — número de casos.

Después le siguen t líneas, la i -ésima línea contiene un número entero n ($1 \leq n \leq 10^9$).

Output:

Imprima t enteros, el i -ésimo entero deberá ser la respuesta para el i -ésimo caso.

Ejemplos:

[3.2] Un número $1 \leq x \leq 10^9$ es elegido. Te darán dos

Entrada	Salida
5	0
1	1
9	1
10	3
34	88005553
880055535	

enteros a y b , los dos divisores más grandes del número x y también se cumple que $1 \leq a < b < x$.

Para los números dados a, b tienes que encontrar el valor de x .

Input:

Cada test contiene varios casos. La primera línea contiene un entero t ($1 \leq t \leq 10^4$), el número de casos. Luego la descripción de los casos.

La única línea de cada caso contiene dos enteros a, b ($1 \leq a < b \leq 10^9$).

Se garantiza que a, b son los divisores más grandes para algún número $1 \leq x \leq 10^9$.

Output:

Para cada caso de prueba, imprime el número x , tal que a y b son los mayores divisores del número x . Si hay varias respuestas, imprime cualquiera de ellas.

Ejemplos:

Entrada	Salida
8	6
2 3	4
1 2	33
3 11	25
1 5	20
5 10	12
4 6	27
3 9	1000000000
2500000000 5000000000	

Capítulo 4

Geometría computacional

En geometría computacional, necesitaremos una forma de representar a los objetos geométricos en código, a los puntos del plano los representaremos como un par de coordenadas de (x, y) con las operaciones usuales aplicadas coordenada a coordenada, un punto y un vector son intercambiables cuando el contexto lo permite.

```
1 typedef double db;
2 struct point {
3     db x,y;
4     point operator+(point p) {
5         return {x + p.x, y + p.y};
6     }
7     point operator-(point p) {
8         return {x - p.x, y - p.y};
9     }
10    point operator*(db a) {
11        return {x*a, y*a};
12    }
13    point operator/(db a) {
```

```

14         return {x/a, y/a};
15     }
16
17 };

```

Como definimos nuestra propia estructura es necesario que también definamos los comparadores para poder decidir si dos puntos son iguales o no.

```

1 bool operator==(point a, point b) {
2     return a.x==b.x && a.y==b.y;
3 }
4 bool operator!=(point a, point b) {
5     return !(a==b);
6 }

```

Nótese que el definir un comparador para decidir si un punto es "menor" a otro, no es tan fácil qué hacer por lo que usualmente dependiendo el caso los ordenaremos, frecuentemente nos convendrá ordenarlos de forma horaria o antihoraria de acuerdo a los vectores que corresponden los puntos, la pregunta ahora consiste en ¿cómo hacer esto?, antes de ver una técnica para hacerlo, recordaremos algunas operaciones que podemos hacer entre puntos.

4.1. Bases

En primer lugar tenemos el producto punto, dados dos puntos este nos ayuda a medir qué tanto dos vectores apuntan a la misma dirección, si el producto punto es positivo, los vectores apuntan a direcciones similares, si es cero, los vectores son perpendiculares y si es negativo, entonces los vectores apuntan en direcciones opuestas, en programación competitiva este se calcula simplemente con las coordenadas.

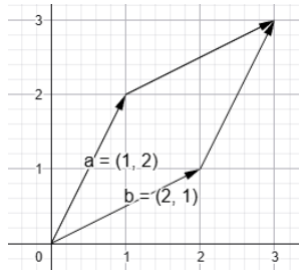

```

1 double prodPunto(point a, point b){
2     return a.x*b.x + a.y*b.y;
3 }

```

Nótese que por como lo calculamos si hacemos el producto punto de un vector consigo mismo tendremos la norma de este al cuadrado.

Otro concepto que nos será relevante recordar es el producto cruz entre dos vectores (denotado $a \times b$), este nos dará el área orientada del paralelogramo que forman dos vectores, con orientada nos referimos a que si calculamos $a \times b$ será el área del paralelogramo negativa, en cambio $b \times a$ nos la dará positiva, esto nos da una noción de si un vector está a la "izquierda" o "derecha" de otro.



la implementación también es sencilla y nos da una forma rápida de comprobar si tres puntos son colineales.

```

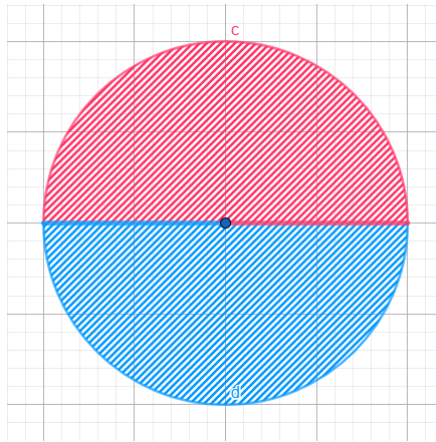
1 double prodCruz(point a, point b) {
2     return a.x * b.y - a.y * b.x;
3 }

```

Con lo anterior ya tenemos las herramientas suficientes para resolver nuestro problema de ordenar los puntos del plano de forma horaria o antihoraria alrededor de un punto.

Supongamos que queremos ordenar los puntos del plano de forma antihoraria respecto al origen y con una referencia de "inicio" que tomaremos como $v = (0, 1)$.

Notemos que si tenemos un conjunto de puntos A , podemos dividir el plano en dos partes, la primera que llamaremos N que tendrá a todos los puntos que estén a la izquierda de v y todos los puntos que sean colineales apuntando a la misma dirección, la segunda se llamará S y serán los restantes, es decir, aquellos a la derecha de v y los colineales que apuntan en dirección opuesta es decir, los puntos en el área roja estarían en N y los del área azul en S .



Entonces de aquí que digamos que para un punto x en A , x está en N si y sólo si se cumple que $v \times x > 0$ o $v \cdot x > 0$ y además $v \times x = 0$, estás siendo las operaciones de producto punto y cruz que vimos arriba, con esto podemos hacer una función que nos verifique si un punto de A está o no en N facilmente

```
1 bool N(point x){
```

```

2   point v = {1,0};
3   return prodCruz(v,x) > 0 || ( prodCruz(v,x)
   == 0 && prodPunto(v,x) > 0 );
4 }

```

Una vez dividido en dos partes notemos que dados x y y , podemos decir que $x < y$ si $x \in N$ y $y \in S$ o ambos están en la misma parte y además $x \times y > 0$, para esto entonces sólo nos basta definir un operador como sigue que es más fácil de manipular, nótese entonces que este ordenamiento tiene complejidad $O(n \log n)$ donde n es la cantidad de puntos en A .

```

1 bool operator <(point x, point y){
2     return (N(x) == N(y) && prodCruz(x,y) > 0)
   || (N(x) && !N(y));
3 }

```

4.2. Intersecciones

En muchas ocasiones nos interesará saber si dados dos segmentos AB y CD estos se intersectan, existen varias formas de revisar esto; sin embargo para evitarnos los muchos casos extremos que pueden aparecer en otros métodos, lo que haremos será pensar que en lugar de segmentos, tenemos dos líneas, la primera parametrizada como $A+t(B-A)$ y la segunda parametrizada como $(x-C) \cdot (D-C)^\perp = 0$ ¹, estas se van a intersectar en un solo punto siempre que no sean paralelas, lo que podemos revisar facilmente verificando que el producto cruz sea diferente a 0, entonces, si no son paralelas, buscaremos el punto de intersección, es decir,

¹Recordemos que dado un vector (x, y) , $(x, y)^\perp = (-y, x)$, en este caso sería $(y_C - y_D, x_D - x_C)$

podemos sustituir la primera recta en la segunda y obtener $(A + t(B - A) - C) \cdot (D - C)^\perp = 0$ de donde podemos despejar t

$$t = \frac{(C - A) \cdot (D - C)^\perp}{(B - A) \cdot (D - C)^\perp}$$

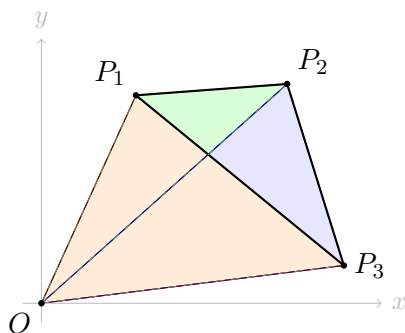
para sustituirlo en $A + t(B - A)$ y obtener el punto de intersección y finalmente checar si este punto está sobre cada uno de los segmentos.

Ejercicio Implementar una función booleana que reciba cuatro puntos A, B, C y D y que nos diga si dos segmentos AB y CD se intersecan.

4.3. Polígonos

Cuando somos más pequeños, se nos enseña que el área de un triángulo se calcula como "base por altura sobre dos", imaginemos que solamente nos dan las coordenadas de los tres puntos que definen un triángulo, para usar lo que nos enseñaron tendríamos que hacer varios calculos para encontrar puntos medios y distancias, nos gustaría encontrar una forma más directa que además se pueda generalizar para polígonos en general.

Supongamos tenemos $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2)$, y $P_3 = (x_3, y_3)$, los tres puntos que delimitan un triángulo, notemos que si consideramos al origen $(0, 0)$ entonces el area buscada equivale a sumar el área de los triángulos P_3OP_2 y P_2OP_1 y restarles el área del triángulo P_3OP_1 .



Estas áreas de los triángulos ya las sabemos calcular, pues recordemos que el producto cruz entre P_3 y P_2 nos da el área con signo del paralelogramo que forman y al dividirlo entre 2, el área del triángulo buscado es de donde deducimos que la formula del área de un triángulo es

$$A(P_1, P_2, P_3) = \frac{|P_1 \times P_2 + P_2 \times P_3 + P_3 \times P_1|}{2}$$

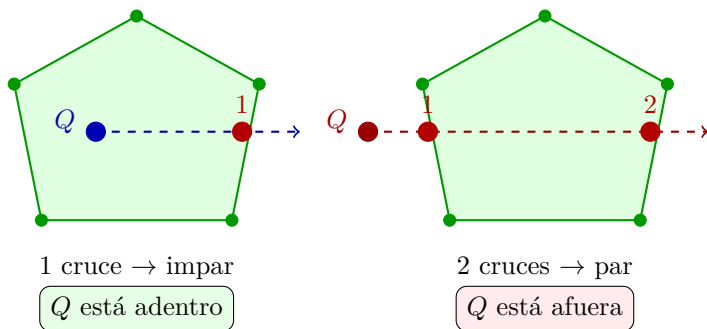
Nótese que no es necesario restar porque el producto cruz ya nos da el área con signo, entonces el signo de menos ya está incluido en la suma.

Resulta que esta misma lógica funciona para polígonos de n vértices donde se suman n triangulos haciendo que el exterior se cancele, la fórmula generalizada entonces quedaría

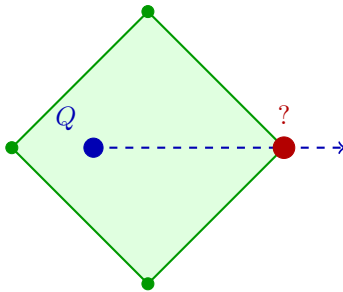
$$A(P_1, \dots, P_n) = \frac{|\sum_{i=1}^n P_i \times P_{i+1}|}{2}$$

con $P_{n+1} = P_1$. Esta fórmula funciona también para polígonos no convexos.

Otro problema que nos podemos encontrar al trabajar con polígonos es dado un punto Q determinar si está afuera o adentro de un polígono dado. El método más usado para resolver este problema se llama "ray casting", la idea es que se lanza un rayo horizontal desde Q hacia la derecha y contamos cuántas veces cruza el polígono, si lo cruza un número impar de veces está adentro, de lo contrario está afuera.



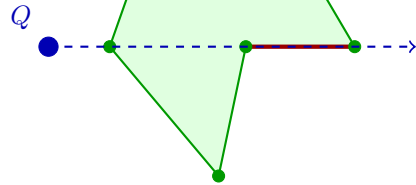
este es un método sencillo para problemas donde sabemos con certeza no pasarán casos extremos, ya que al lanzar el rayo podría suceder lo siguiente



¿1 cruce o 2 cruces?

rayo pasa por un vértice

ambiguo

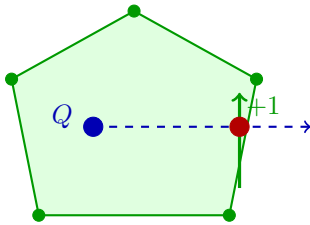


¿cuántos cruces con la arista?

arista horizontal

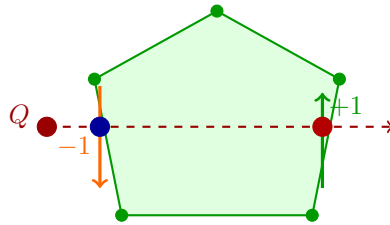
ambiguo

Para resolver este problema, usamos un algoritmo que se llama "winding number", consiste en la misma idea de lanzar un rayo y contar intersecciones, pero en este se va a contar su orientación, nótese que en la primera la arista que intersecta va subiendo entonces se suma 1, en el segundo caso, se resta 1 porque la arista va bajando, si el resultado es diferente de 0, el punto está adentro.



winding = $+1 \neq 0$

Q está adentro



winding = $-1 + 1 = 0$

Q está afuera

+1 arista cruza el rayo de abajo hacia arriba

-1 arista cruza el rayo de arriba hacia abajo

Nótese que la dirección de cada arista que cruza el rayo se determina en dos pasos: primero verificamos que los extremos de la arista estén en lados opuestos del nivel $y = y_Q$ usando las condiciones sobre las coordenadas y , y luego usamos $\text{prodCruz}(P_{i+1} - P_i, Q - P_i)$ para saber si Q queda a la izquierda o derecha de la arista, lo que determina si el cruce suma $+1$ o -1 .

Ahora, el lector podrá preguntarse qué sucede con el caso donde el rayo intersecta exactamente en un vértice P_i . En este algoritmo las condiciones para que una arista cuente son asimétricas: para la arista $P_{i-1}P_i$ que *llega* a P_i se requiere

$$y_{P_{i-1}} \leq y_Q < y_{P_i}$$

como $y_{P_i} = y_Q$, la condición estricta $<$ no se satisface y la arista **no cuenta**. Para la arista P_iP_{i+1} que *sale* de P_i se requiere

$$y_{P_i} \leq y_Q < y_{P_{i+1}}$$

como $y_{P_i} = y_Q$, la condición $\leq$ sí se satisface, y la arista **cuenta** únicamente si $y_{P_{i+1}} > y_Q$, es decir, si la arista realmente cruza el rayo hacia arriba.

El mismo razonamiento elimina las aristas horizontales: si $y_{P_i} = y_{P_{i+1}} = y_Q$, ninguna de las dos condiciones se satisface y la arista simplemente no se cuenta, ya explicado el algoritmo, la implementación quedaría:

```

1 int winding_number(vector<point> poly, point q)
  {
2     int wn = 0;
3     int n = poly.size();
4     for (int i = 0; i < n; i++) {
5         point a = poly[i], b = poly[(i+1) % n];

```



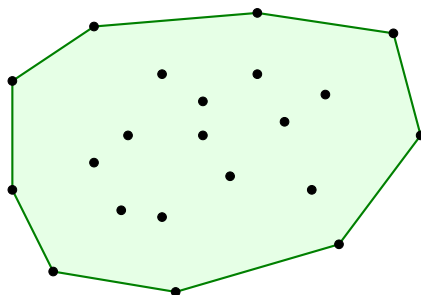
```

6         if (a.y <= q.y) {
7             // arista cruza hacia arriba: b esta
            estrictamente sobre el rayo
8             if (b.y > q.y && prodCruz(b-a,q-a) >
                0)
9                 wn++;
10        } else {
11            // arista cruza hacia abajo: b esta
            sobre o bajo el rayo
12            if (b.y <= q.y && prodCruz(b-a,q-a)
                < 0)
13                wn--;
14        }
15    }
16    return wn;
17 }
18
19 bool dentro(vector<point> poly, point q) {
20     return winding_number(poly, q) != 0;
21 }

```

4.4. Convex Hull

Dado un conjunto de puntos $S \subset \mathbb{R}^2$, su **cápsula convexa** (*convex hull*) intuitivamente es la figura que se formaría si colocamos una liga estirada alrededor de todos los puntos, más formalmente es el polígono convexo más pequeño que los contiene a todos. En la imagen siguiente S son todos los puntos negros y el hull sería es el polígono dibujado de verde.



Existen varios algoritmos para calcularlo. El más conocido es **Graham Scan**; sin embargo, este ordena los puntos por ángulo por lo que hay que tener cuidado con los casos delicados cuando varios puntos tienen el mismo ángulo polar. Por ello preferimos **Andrew's Monotone Chain**, ambos algoritmos tienen la misma complejidad $O(n \log n)$, misma idea para construir el convex hull, pero ordena por coordenada x en lugar de ángulo, esto lo hace más fácil de implementar sin errores en un concurso.

Nótese que para saber qué polígono es el hull, sólo tenemos que saber qué puntos forman parte de él y en qué orden. El algoritmo de Andrew construye el convex hull en dos pasadas sobre los puntos ordenados por x , primero se construye la parte inferior y luego la superior.

Para la parte superior se van a recorrer los puntos de izquierda a derecha, y tendremos un vector *hull* con la característica que los últimos tres puntos siempre van a tener producto cruz es positivo, es decir dados A, B, C tenemos que $(B - A) \times (C - A) > 0$, inicialmente se agregan los dos primeros puntos a *hull* y después cada que lleguemos a un nuevo punto P si este cumple la condición se agrega, de lo contrario, se descarta el tope de la pila y después se

agrega P a *hull*. Para la parte inferior es exactamente el mismo procedimiento con la única diferencia que se recorre de derecha a izquierda. La implementación quedaría:

```
1 vector<point> convex_hull(vector<point> pts) {
2     int n = pts.size();
3     if (n < 2) return pts;
4     sort(pts.begin(), pts.end());
5     vector<point> hull;
6     /*construimos hull inferior
7     izquierda a derecha*/
8     for (int i = 0; i < n; i++) {
9         while (hull.size() >= 2 &&
10             /*producto cruz entre
11             B-A y C-A*/
12             prodCruz(hull.back()-hull[hull.size()-2], pts[i]-hull[hull.size()-2])<= 0)
13             hull.pop_back();
14         hull.push_back(pts[i]);
15     }
16
17     /*construimos hull superior
18     derecha a izquierda*/
19     int lower_size = hull.size() + 1;
20     for (int i = n-2; i >= 0; i--) {
21         while (hull.size() >= lower_size &&
22             /*producto cruz entre
23             B-A y C-A*/
24             prodCruz(hull.back()-hull[hull.size()-2], pts[i]-hull[hull.size()-2])<= 0)
25             hull.pop_back();
26         hull.push_back(pts[i]);
27     }
28
29     //al cerrarlo este punto se repite
30     hull.pop_back();
31     return hull;
32 }
```

La desigualdad ≤ 0 descarta también los puntos colinea-

les, de modo que el hull resultante solo contiene los vértices extremos. Si el problema requiere incluir los puntos colineales sobre las aristas del hull, podemos hacer la desigualdad estricta.

Ejercicio Dado un conjunto de n puntos, encontrar el par de puntos más lejanos (diámetro).

4.5. Algoritmos de barrido

El barrido de línea no es por si solo un algoritmo, si no una idea general detrás de varios algoritmos. La idea es mover una línea vertical de izquierda a derecha sobre el plano y procesar los objetos geométricos conforme se van encontrando en lugar de considerar todos a la vez, a esta acción de procesamiento se le llama evento. Es decir, que en cada momento solo nos importa lo que está tocando la línea, para esto es necesario mencionar que la línea no se mueve continuamente si no que salta de evento en evento.

Estos algoritmos usualmente tienen tres partes:

- **Cola de eventos** esta es una lista ordenada por x de los eventos.
- **Estructura de incidencia** es una estructura que nos dirá qué objetos intersectan la línea en este momento, la elección de la estructura a usar depende el problema, pueden usarse sets, multisets, segment trees, colas, entre otras.
- **Procesamiento de eventos** es la lógica con la que procesaremos cada que ocurra un evento, esta de-

pende mucho de qué problema particular querramos resolver.

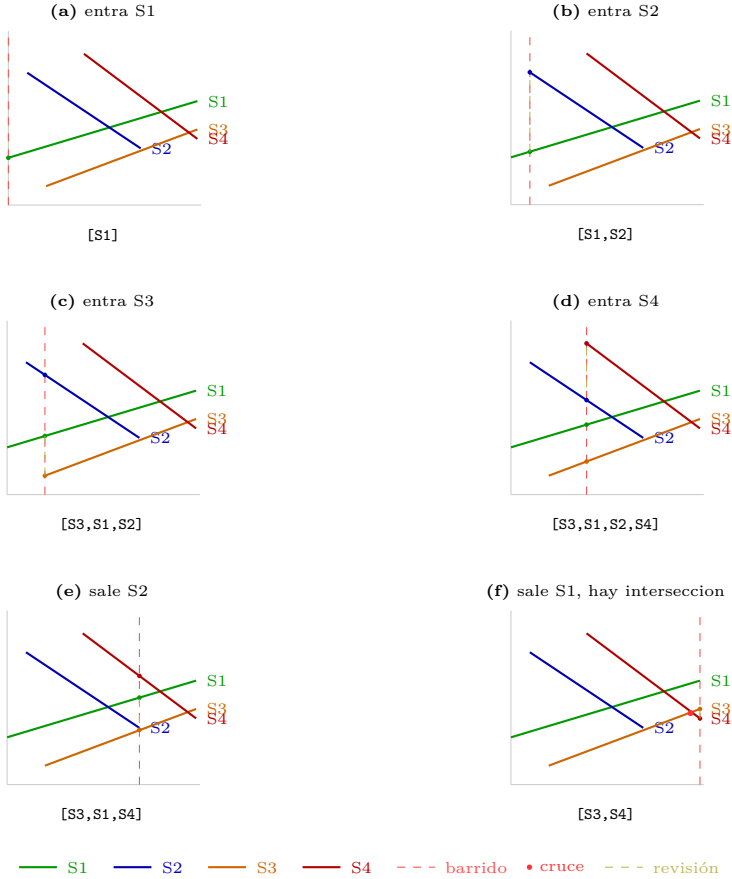
Propongamos el siguiente problema: Dados n segmentos en el plano, decir si al menos dos de ellos se intersectan.

Podríamos pensar ingenuamente que la única forma de resolver este problema es revisar todos los pares de segmentos y revisar intersección entre ellos es decir una complejidad de $O(n^2)$, sin embargo, usando barrido de línea podemos resolver este problema más rápido.

El algoritmo que veremos para resolver este problema se llama *Shamos – Hoey* y se basa en que si dos segmentos se llegaran a intersectar existe un momento durante el barrido en el que son vecinos en la estructura ordenada por y . Esto hace que el problema se reduzca a que en lugar de revisar todos los pares, revisar los vecinos en la estructura cada que suceda un evento.

Los eventos que podemos tener en este problema son puntos que indican el inicio o final de un segmento, entonces tendremos dos tipos de eventos:

- **Inicio** Insertaremos el punto a la estructura que ya dijimos está ordenada por y y verificaremos si se intersecta con el vecino superior o inferior.
- **Final** Este evento indica el fin de un segmento, entonces lo quitaremos de la estructura, pues ya no nos interesa. Sin embargo, hay que tener cuidado es necesario revisar si los dos vecinos (que al quitar el punto se vuelven adyacentes) se intersectan.



Si en cualquier punto encontramos una intersección, entonces terminamos el algoritmo pues ya resolvimos el problema. Es importante mencionar, que si hay varios eventos en un mismo x , primero procesaremos los inicios y después los finales, así lograremos también comprobar intersecciones si los segmentos comparten vértices.

La estructura que nos convendrá para este problema es un `set<pair<point,point>>` de C++ ordenado por la coordenada y de cada segmento en la posición actual x de la línea. El comparador para ordenar evalúa para cada segmento

$$y(x) = y_1 + \frac{(y_2 - y_1)(x - x_1)}{x_2 - x_1}$$

al implementarlo hay que ser cuidadosos pues como estaremos trabajando con decimales al hacer comparaciones debemos agregar una tolerancia ε .

```
1 //representamos segmentos como pares de puntos
2 using seg = pair<point, point>;
3 //funcion que nos devuelve dado un segmento y un
  x, la y del segmento.
4 db getY(const seg& s, db x) {
5     if (abs(s.first.x - s.second.x) < 1e-9){
6         return s.first.y;
7     }
8     return s.first.y +
9         (s.second.y - s.first.y)
10        * (x - s.first.x)
11        / (s.second.x - s.first.x);
12 }
13
14 db sweepX;
15 //Comparador para ordenar los segmentos por y
16 struct cmp {
17     bool operator()(const seg& a, const seg& b)
18     const {
19         db ya = getY(a, sweepX);
20         db yb = getY(b, sweepX);
21         if (abs(ya - yb) > 1e-9){
22             return ya < yb;
23         }
24         return a < b;
25     }
26 };
```

```

27 bool shamos_hoey(vector<seg> segs) {
28     int n = segs.size();
29     /*asegurarse que los segmentos
30     vayan de izquierda a derecha*/
31     for (auto& s : segs){
32         if (s.first.x > s.second.x){
33             swap(s.first, s.second);
34         }
35     }
36     /*
37     eventos guardara la x del punto,
38     1 o -1 que nos indicara si es de inicio
39     o fin respectivamente,
40     el indice en el vector original
41     */
42     vector<tuple<db,int,int>> eventos;
43     for (int i = 0; i < n; i++) {
44         eventos.push_back(
45             {segs[i].first.x, 1, i}
46         );
47         eventos.push_back(
48             {segs[i].second.x, -1, i}
49         );
50     }
51     sort(eventos.begin(), eventos.end(),
52     [](auto& a, auto& b){
53         if (abs(get<0>(a)
54             - get<0>(b)) > 1e-9){
55             return get<0>(a) < get<0>(b);
56         }
57         return get<1>(a) > get<1>(b);
58     });
59
60     set<seg,cmp> activos;
61     /*estructura que nos ayude
62     a borrar sin depender del
63     comparador que definimos*/
64     vector<set<seg,cmp>::iterator> pos(n,
65     activos.end());
66     for (auto& [x, tipo, id] : eventos) {
67         sweepX = x;

```



```

67         //procesamos nodo de inicio
68         if (tipo == 1) {
69             auto it =
70                 activos.insert(segs[id]).first;
71             pos[id] = it;
72             auto sig = next(it);
73             auto ant = (it != activos.begin())
74                 ? prev(it)
75                 : activos.end();
76             /*buscar interseccion con
77             vecinos, se dejo como ejercicio
78             en intersecciones*/
79             if (sig != activos.end()
80                 && intersectan(it, sig)){
81                 return true;
82             }
83             if (ant != activos.end()
84                 && intersectan(it, ant)){
85                 return true;
86             }
87             //procesamos nodo de final
88         } else {
89             auto it = pos[id];
90             auto sig = next(it);
91             auto ant = (it != activos.begin())
92                 ? prev(it)
93                 : activos.end();
94             if (sig != activos.end()
95                 && ant != activos.end()
96                 && intersectan(*sig, *ant)){
97                 return true;
98             }
99             activos.erase(it);
100         }
101     }
102     return false;
103 }

```

Nótese que hay $2n$ eventos y cada evento realiza a lo más dos chequeos de intersección que cada uno tiene compleji-

dad de $O(1)$ y una adición o eliminación con complejidad $O(\log n)$. Además ordenar inicialmente los segmentos tiene complejidad $O(n \log n)$ entonces la complejidad del algoritmo es $O(n \log n)$ que en efecto es más rápido que $O(n^2)$.

En el problema que resolvimos bastaba con encontrar una intersección, podríamos preguntarnos qué sucede si queremos encontrar todas las intersecciones. El algoritmo que se usa para este caso se llama **Bentley-Ottmann** que encuentra todas las intersecciones en $O((n+k)\log n)$ donde k es el número de intersecciones. Podría parecer que la diferencia es sutil y sólo basta con seguir el algoritmo que estábamos usando antes sin detenerlo, pero recordemos que nuestro algoritmo dependía fuertemente en el orden de los segmentos para encontrar intersecciones, cuando sucede una intersección, el orden de dos segmentos se cambia esto provoca que cada que encontremos un cruce tendremos que cambiar el orden de la estructura y además revisar si los nuevos vecinos tienen cruces. Y no solo eso, si no que cada que agreguemos un segmento si encontramos una intersección tendremos que calcularla y agregarla a la cola de eventos.

Mayores complicaciones de Bentley-Ottmann

- Cuando dos segmentos cambian de orden, nuestro **set** ordenado por y ya no es correcto, pues el comparador de este cambia con el tiempo dependiendo de x , tendríamos que manualmente extraer e insertar los elementos necesarios.
- Si hay casos degenerados con tres segmentos o más que se intersectan en el mismo punto, hay que ser muy cuidadosos en que orden se procesan y cómo actualizamos la estructura usada.

- Si no sabemos con certeza una cota superior para la cantidad de intersecciones y se dé el peor caso (n^2), puede suceder que el algoritmo con complejidad $O(n^2 \log n)$ sea más lento que la fuerza bruta que era $O(n^2)$.

Usualmente no se necesita implementar Bentley Oattmann en concursos, ya que su implementación son alrededor de 200 líneas con casos delicados de manejar, sería complicado encontrar bugs bajo presión de tiempo, además que los problemas donde se requieren todas las intersecciones usualmente permiten fuerza bruta. Lo más usado es la idea en sí misma aplicado a problemas más sencillos como la unión de área de rectángulos como veremos en los ejemplos a continuación.

4.6. Ejercicios

4.7. Problemas